Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · WEEK 10

Synchronization II

The classical problems solved with semaphores and monitors, and the synchronization tools of Windows, Linux, POSIX and Java.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Explain the bounded-buffer, readers–writers and dining-philosophers problems.
2. Solve each of them with semaphores or a monitor.
3. Describe the synchronization tools used inside Windows and Linux.
4. Use POSIX and Java to solve synchronization problems.
5. Name alternatives to locks.

THIS WEEK

Topic 8

Week 9 gave us semaphores and monitors. This week we use them on the three problems every scheme is tested against.

DISCUSSION

Five people, five chopsticks, and everyone is hungry

Each person needs two chopsticks to eat, and picks up the left one first. What happens if all five reach at the same moment?

PART 1

The bounded-buffer problem

The producer and the consumer of Week 3, this time done correctly.

Three semaphores



1

mutex

A binary semaphore. Only one process changes the buffer at a time.

3

full

Counts the full slots. Starts at 0. The consumer waits on it.

5

empty

Counts the empty slots. Starts at n, here 8. The producer waits on it.

A buffer of n = 8 slots holding three items. At every moment, full + empty = n.

Producer and consumer

```
/* producer */
while (true) {
    /* produce an item */

    wait(empty);
    wait(mutex);
    /* add the item to the buffer */
    signal(mutex);
    signal(full);
}
```

```
/* consumer */
while (true) {
    wait(full);
    wait(mutex);
    /* remove an item from the buffer */
    signal(mutex);
    signal(empty);

    /* consume the item */
}
```

The two are mirror images. The producer makes full slots for the consumer, and the consumer makes empty slots for the producer.

The order of the two waits matters

ROOM FIRST, THEN THE LOCK

Correct

wait(empty), then wait(mutex). The producer first makes sure there is room, and only then locks the buffer.

THE LOCK FIRST

Deadlock

wait(mutex), then wait(empty). If the buffer is full, the producer sleeps while holding the lock. The consumer can never get in to make room.

PART 2

The readers–writers problem

Shared data that many processes read and a few change.

Readers and writers



Readers

They only read the data set. They change nothing, so any number of them may read at the same time.



Writers

They can both read and write. A writer must have the data set to itself: no other writer, and no reader.

A plain mutex would be correct but wasteful: it would make readers wait for each other for no reason.

The shared variables

SEMAPHORE

rw_mutex

A semaphore initialised to 1. Held by a writer, or by the readers as a group.

SEMAPHORE

mutex

A semaphore initialised to 1. It protects read_count.

INTEGER

read_count

An integer initialised to 0: how many processes are reading now.

Writer and reader

```
/* writer */
while (true) {
    wait(rw_mutex);

    /* write */

    signal(rw_mutex);
}
```

```
/* reader */
while (true) {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex); /* first reader in */
    signal(mutex);

    /* read */

    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex); /* last reader out */
    signal(mutex);
}
```

Two variations, and starvation

READERS FIRST

First variation

No reader waits unless a writer already has the data. This is the solution above. A steady stream of readers can starve the writers.

WRITERS FIRST

Second variation

Once a writer is ready, it writes as soon as possible. No new reader may start. Now the readers can starve.

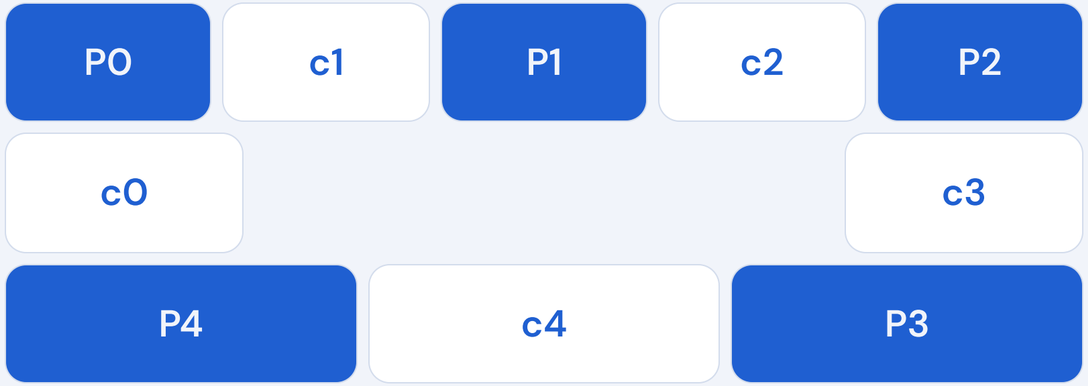
Some kernels provide reader–writer locks, which solve the problem directly. They are worth their cost when the data is read often and changed rarely.

PART 3

The dining-philosophers problem

Several processes, several resources, and each process needs more than one.

The problem



Five philosophers P0 to P4 around a table, with one chopstick between each pair.

A philosopher thinks, and from time to time gets hungry.

To eat, a philosopher needs both the left and the right chopstick, and picks them up one at a time.

After eating, both chopsticks are put down and thinking resumes.

It stands for any system where processes need several shared resources at once.

A semaphore for each chopstick

```
semaphore chopstick[5]; /* all 1 */

/* philosopher i */
while (true) {
    wait(chopstick[i]);
    wait(chopstick[(i + 1) % 5]);

    /* eat */

    signal(chopstick[i]);
    signal(chopstick[(i + 1) % 5]);

    /* think */
}
```

No two neighbours eat at the same time. Mutual exclusion holds.

But this answers the opening question badly. If all five become hungry together and each takes the left chopstick, every semaphore is 0.

Each then waits for the right chopstick for ever. Deadlock.

Three ways to avoid the deadlock

01

One seat fewer

Allow at most four philosophers to sit at the table at the same time.

02

Both or neither

A philosopher picks up the chopsticks only if both are available, and does so inside a critical section.

03

Break the symmetry

An odd-numbered philosopher picks up the left chopstick first. An even-numbered one picks up the right first.

A solution free of deadlock may still allow starvation: one philosopher might never get both chopsticks.

A monitor solution

```
monitor Dining {
    enum {THINKING, HUNGRY, EATING} state[5];
    condition self[5];

    procedure pickup(int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING)
            self[i].wait();
    }

    procedure putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5);
        test((i + 1) % 5);
    }
}
```

test(i)

If philosopher i is hungry and neither neighbour is eating, i starts eating and is signalled.

pickup and putdown

A philosopher who cannot eat waits on a private condition. On putting down, it lets each neighbour try.

What the monitor guarantees



No deadlock

A philosopher takes both chopsticks in one step, inside the monitor, or takes none.



Mutual exclusion

A philosopher eats only when neither neighbour is eating.



Starvation is possible

Two neighbours can take turns eating so that the one between them never gets the chance.

PART 4

Synchronization inside the kernel

How Windows and Linux protect their own data.

Week 10 · 19

Windows

On one processor

The kernel protects global resources by masking interrupts for a short time.

On several processors

It uses spinlocks. A thread holding a spinlock is never preempted.

Outside the kernel

Dispatcher objects: mutex locks, semaphores, events, which act like condition variables, and timers.

Two states

A dispatcher object is signaled, meaning available, or non-signaled, meaning a thread that asks for it will block.

Linux

Tool	What it is for
Atomic integers	Simple updates of a counter without a lock: atomic_t and its operations
Spinlocks	Short critical sections on multiprocessor systems
Mutex locks	Longer critical sections, where the waiting task should sleep
Semaphores and reader–writer locks	Counting, and data that is read much more often than it is written

On a single-processor system a spinlock would be pointless, so the kernel instead disables and enables kernel preemption. The kernel has been fully preemptive since version 2.6.

PART 5

POSIX synchronization

The tools available to our own C programs on Linux and UNIX.

Week 10 · 22

POSIX mutex locks

```
#include <pthread.h>

pthread_mutex_t m;
pthread_mutex_init(&m, NULL);

pthread_mutex_lock(&m);
/* critical section */
pthread_mutex_unlock(&m);
```

The basic tool with Pthreads.

A thread that calls pthread_mutex_lock() while the lock is held is blocked until the owner unlocks it.

Each function returns 0 on success, and an error code otherwise.

POSIX semaphores

```
#include <semaphore.h>

/* named: shared by name */
sem_t *s = sem_open("/gate", O_CREAT, 0666, 1);
sem_wait(s);
/* critical section */
sem_post(s);

/* unnamed: shared by address */
sem_t u;
sem_init(&u, 0, 1);
sem_wait(&u);
/* critical section */
sem_post(&u);
```

Named

Created with a name, so unrelated processes can open the same semaphore.

Unnamed

Used by the threads of one process, or by processes that share the memory it is in.

POSIX calls the two operations sem_wait() and sem_post().

POSIX condition variables

```
pthread_mutex_t m;
pthread_cond_t c;

/* the thread that waits */
pthread_mutex_lock(&m);
while (!ready)
    pthread_cond_wait(&c, &m);
pthread_mutex_unlock(&m);

/* the thread that signals */
pthread_mutex_lock(&m);
ready = true;
pthread_cond_signal(&c);
pthread_mutex_unlock(&m);
```

C has no monitors, so a condition variable is paired with a mutex.

pthread_cond_wait() releases the mutex and puts the thread to sleep. When the thread wakes, it holds the mutex again.

The condition is tested in a while loop, not an if: after waking, the thread checks that the condition is really true.

PART 6

Java synchronization and alternatives

A language with monitors built in, and some ways to avoid locks altogether.

Week 10 · 26

Java monitors

```
public class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }

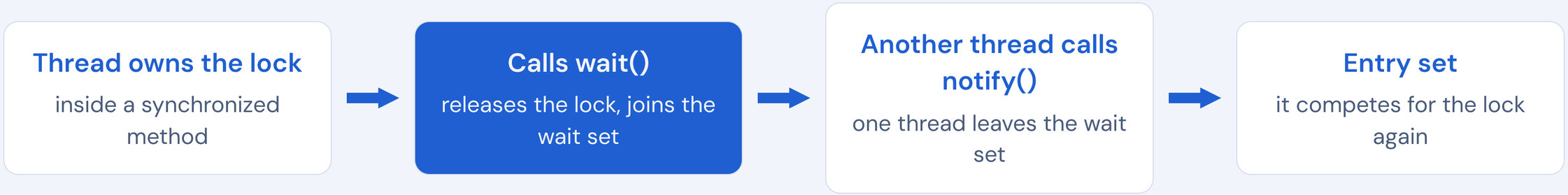
    public synchronized int get() {
        return count;
    }
}
```

Every Java object has a single lock.

A thread that calls a synchronized method must own the object's lock. If another thread owns it, the caller is placed in the object's entry set.

The lock is released when the method returns.

wait() and notify()



notify() picks an arbitrary thread from the wait set. notifyAll() moves them all. A woken thread must test its condition again, in a loop.

Locks, semaphores and conditions in Java



ReentrantLock

Like a synchronized block, with extra features such as a fairness option. The unlock() goes in a finally clause, so the lock is released even if an exception occurs.



Semaphore

A counting semaphore: acquire() and release().



Condition

A condition variable tied to a ReentrantLock, created with newCondition(). Its operations are await() and signal(). A lock may have several conditions.

Alternative approaches

01

Transactional memory

A sequence of reads and writes of memory is marked as atomic. The system runs it as a transaction: if it conflicts with another, it is rolled back and retried. No locks, so no deadlock.

02

OpenMP

The directive that marks a critical section tells the compiler that only one thread at a time may run the block that follows.

03

Functional languages

Languages such as Erlang and Scala do not keep state that can change. Without mutable shared data, race conditions cannot arise.

Key takeaways

01

Bounded buffer

Three semaphores: mutex for the buffer, full and empty for the slots. Take the slot before the lock.

02

Readers and writers

Readers share, writers exclude. Every simple solution can starve one side.

03

Dining philosophers

Taking resources one at a time, all in the same order around a circle, leads to deadlock.

04

The same tools everywhere

Mutex locks, semaphores and condition variables, in kernels, in POSIX and in Java.

Next session

COMING UP

Deadlock

The four necessary conditions, resource-allocation graphs, prevention, avoidance with the banker's algorithm, detection and recovery.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 7.